

API, Authentication, Prompting

A Comprehensive 7-Page Plain-Text Reference Guide

1. What is an API?

An **API (Application Programming Interface)** is a set of rules, protocols, and tools that allows different software applications to communicate with each other. It defines how requests and responses should be structured so that one program can ask another program to perform a task or share data, without needing to know the internal workings of that other program.

Analogy:

Think of an API as a waiter in a restaurant. You (the client) look at the menu (the API documentation), place an order (make a request), and the waiter takes your order to the kitchen (the server) and brings back your food (the response). You never need to go into the kitchen yourself.

Key Characteristics of an API:

- **Interface Contract:** Defines inputs, outputs, and expected behaviour between systems.
- **Abstraction:** Hides complex internal logic; the caller only needs to know what to send and what to expect back.
- **Reusability:** One API can serve many clients — web apps, mobile apps, third-party services.
- **Interoperability:** Enables systems built in different languages/platforms to work together.
- **Scalability:** Backend services can be updated independently without breaking the contract.

Common API Types:

REST (Representational State Transfer): Uses HTTP methods (GET, POST, PUT, DELETE). Most widely used. Stateless.

SOAP (Simple Object Access Protocol): XML-based, strict contract via WSDL. Used in enterprise/legacy systems.

GraphQL: Query language for APIs. Client specifies exactly what data it needs.

gRPC: High-performance RPC framework by Google. Uses Protocol Buffers.

WebSocket: Full-duplex communication channel over a single TCP connection. Used for real-time apps.

How an API Request Works (REST Example):

1. Client sends an HTTP request to an endpoint URL, e.g. GET <https://api.example.com/users/42> 2. The server processes the request, queries a database if needed, and prepares a response. 3. The server returns an HTTP response with a status code (200 OK, 404 Not Found, etc.) and a body (usually JSON or XML). 4. The client parses the response and uses the data.

API Use Cases:

- Payment gateways (Stripe, Razorpay) integrated into e-commerce sites.
- Weather data fetched from OpenWeatherMap and shown in mobile apps.
- Login with Google / Facebook using their OAuth APIs.
- Sending automated emails via SendGrid or Mailgun APIs.
- AI capabilities via Anthropic or OpenAI APIs.

2. Overview of Local Variables

Definition:

A **local variable** is a variable that is declared inside a function, block, or scope and is only accessible within that specific scope. It is created when the scope is entered and destroyed when the scope is exited. It cannot be accessed from outside the block where it was defined.

Purpose of Local Variables:

- **Encapsulation:** Keeps data contained within a function, preventing unintended access or modification from other parts of the program.
- **Memory Efficiency:** Local variables are allocated on the stack and deallocated automatically when the function returns.
- **Avoiding Name Conflicts:** Two functions can have local variables with the same name without interfering with each other.
- **Readability and Maintainability:** Code is easier to understand when variables have limited, well-defined scope.
- **Thread Safety:** Local variables are private to each function call, so concurrent threads do not share them by default.

Usage in Different Contexts:

In Programming Languages (Python, Java, C, JavaScript): Any variable declared inside a function body is local. Example in Python:

```
def calculate_area(radius): | pi = 3.14159 | area = pi * radius * radius | return area # 'pi' and 'area' are local variables — inaccessible outside this function.
```

In APIs and Configuration:

- **Environment variables vs local config:** Local variables in API contexts often refer to request-scoped data — headers, parsed tokens, or temporary computation results that exist only during that request cycle.
- **Middleware scope:** In frameworks like Express.js, `res.locals` holds local variables scoped to a single request-response cycle.
- **Template engines:** Variables passed to a template are local to that render call.

Local vs Global vs Closure Variables:

- **Local:** Declared Inside a function/block. Scope: Only within that scope. Lifetime: Function ends.
- **Global:** Declared Outside all functions. Scope: Everywhere in the file/module. Lifetime: Program ends.
- **Closure:** Declared Outer function captured by inner. Scope: Inner function retains reference. Lifetime: No more references.

Best Practices:

- Prefer local variables over globals to reduce side effects.
- Use descriptive names even for short-lived variables.
- Initialise variables before use to avoid undefined behaviour.
- Limit variable scope to the smallest block that needs it.

3. Types of Authentication

Authentication is the process of verifying the identity of a user, device, or system trying to access a resource. It answers the question: *'Who are you?'* — before granting access.

3.1 Basic Authentication:

Credentials (username and password) are encoded in Base64 and sent in the HTTP Authorization header on every request. Simple to implement but insecure unless used over HTTPS. Rarely recommended for modern APIs.

3.2 API Key Authentication:

A unique string (key) is issued to a client and sent with each request (in a header, query parameter, or body). Easy to use but the key must be kept secret. Common in public APIs like Google Maps and OpenAI.

3.3 Token-Based Authentication (Bearer Tokens / JWT):

After login, the server issues a token (often a JWT — JSON Web Token). The client sends this token in the Authorization header as *'Bearer {token}'*. The token is self-contained: it carries user info, expiry time, and a signature. No session stored on the server (stateless). Widely used in REST APIs and Single Page Applications.

3.4 Session-Based Authentication:

The server creates a session after login and stores it server-side. A session ID is sent to the client as a cookie. On subsequent requests, the browser automatically sends the cookie. The server looks up the session. Used in traditional web apps (PHP, Django, Rails).

3.5 Multi-Factor Authentication (MFA):

Combines two or more factors: something you know (password), something you have (OTP via SMS/app), something you are (biometrics). Greatly increases security. Widely used in banking, email providers, and enterprise apps.

3.6 Error Handling in Authentication:

Proper error codes and messages are critical for security and usability:

- **400 Bad Request:** Malformed request, missing required fields.
- **401 Unauthorized:** No credentials provided or credentials are invalid.
- **403 Forbidden:** Credentials valid but user lacks permission for this resource.
- **429 Too Many Requests:** Rate limit exceeded — too many login attempts.
- **500 Internal Server Error:** Server-side failure during authentication process.

Security Note: Never reveal whether the username or password was wrong in error messages — always say 'Invalid credentials' to prevent user enumeration attacks.

3.7 OAuth 2.0 — Open Authorisation

OAuth 2.0 is an industry-standard authorisation framework that allows a third-party application to obtain limited access to a user's account on another service, without exposing the user's credentials. It is used by Google, Facebook, GitHub, and virtually every major platform for 'Login with...' functionality.

Core Roles in OAuth 2.0:

- **Resource Owner:** The end user who owns the data.
- **Client:** The third-party application requesting access.
- **Authorization Server:** Issues access tokens after authenticating the user (e.g. Google's auth server).
- **Resource Server:** Hosts the protected resources/API (e.g. Google Drive API).

OAuth 2.0 Grant Types (Flows):

- **Authorization Code (most secure):** Used by server-side web apps. User is redirected to the auth server, grants permission, and an authorization code is exchanged for an access token.
- **PKCE (Proof Key for Code Exchange):** Extension of Authorization Code for public clients (SPAs, mobile apps). Prevents code interception attacks.
- **Client Credentials:** Machine-to-machine (no user involved). Client authenticates with its own credentials to get an access token.
- **Implicit (deprecated):** Token returned directly in the URL fragment. No longer recommended due to security risks.
- **Device Authorization:** For devices with limited input (smart TVs). User approves on a secondary device.

OAuth 2.0 Authorization Code Flow — Step by Step:

1. User clicks 'Login with Google' on your app.
2. App redirects user to Google's Authorization Server with `client_id`, `redirect_uri`, `scope`, and `state`.
3. User logs in on Google and grants permission.
4. Google redirects back to your app with an authorization code.
5. Your server exchanges the code for an access token (and optionally a refresh token) by calling Google's token endpoint.
6. Your app uses the access token to call the Google API on behalf of the user.
7. When the token expires, use the refresh token to get a new access token without re-prompting the user.

Key Tokens in OAuth:

- **Access Token:** Short-lived (minutes to hours). Used to access protected resources.
- **Refresh Token:** Long-lived. Used to obtain a new access token when the current one expires.
- **ID Token (OpenID Connect):** JWT containing user identity info. Used for authentication on top of OAuth.

4. Prompting Techniques

Prompting is the art of crafting inputs to a large language model (LLM) to guide it toward producing accurate, relevant, and useful outputs. The quality of the prompt directly determines the quality of the response.

4.1 Zero-Shot Prompting:

The model is given a task with no examples. It relies entirely on its training. Best for simple, well-defined tasks.

Example: "Translate this sentence to French: The sky is blue."

4.2 One-Shot Prompting:

One example is provided to set the pattern. Helps the model understand the format expected.

Example: "Q: Capital of France? A: Paris. Q: Capital of Japan? A:"

4.3 Few-Shot Prompting:

Two to five examples are provided. The model infers the task pattern from the examples. Highly effective for classification, extraction, or format-specific tasks.

4.4 Chain-of-Thought (CoT) Prompting:

The model is asked to reason step-by-step before giving an answer. Dramatically improves performance on math, logic, and multi-step reasoning tasks. Triggered by phrases like 'think step by step' or 'explain your reasoning'.

Example: "If John has 5 apples and gives away 2, how many remain? Think step by step."

4.5 Role / Persona Prompting:

Assigning a role to the model shapes its tone, vocabulary, and perspective. Useful for domain-specific tasks.

Example: "You are a senior Python developer. Review this code for bugs."

4.6 Instruction Prompting:

Direct, explicit instructions in natural language. Works well with instruction-tuned models (GPT-4, Claude, Gemini). Specify format, length, tone, and constraints clearly.

4.7 Contextual Prompting:

Background context is provided before the task. Helps the model tailor the response to a specific situation, domain, or user profile.

4.8 System Prompt:

In APIs like Anthropic and OpenAI, a system prompt sets persistent instructions, persona, and constraints for the entire conversation. It is injected before user messages and cannot be overridden by the user in well-configured deployments.

4.9 Retrieval-Augmented Generation (RAG) Prompting:

Relevant documents or chunks are retrieved from a knowledge base and injected into the prompt as context before asking the question. Reduces hallucination and keeps answers grounded in specific, up-to-date sources.

4.10 Tree-of-Thought (ToT) Prompting:

The model explores multiple reasoning paths simultaneously, evaluates them, and selects the best one. More powerful than linear chain-of-thought for complex problem solving.

5. Existing Short / Concise Types of Prompts

Shorter prompt types are designed for efficiency — minimal tokens, maximum clarity. They are used in production systems, chatbots, and constrained API environments.

5.1 Direct Question Prompts:

A single, clear question with no padding. Best when the model already has context.

"What is the boiling point of water at sea level?"

5.2 Imperative / Command Prompts:

Start with a verb. Specify the action and output format concisely.

"Summarise this in 3 bullet points." | "List 5 Python best practices."

5.3 Fill-in-the-Blank Prompts:

Provide an incomplete statement for the model to complete.

"The main cause of World War I was ____."

5.4 Yes/No Prompts:

Binary-answer prompts for classification or validation.

"Is Python a statically typed language? Answer yes or no."

5.5 Comparative Prompts:

Ask the model to compare two items concisely.

"Difference between REST and GraphQL in two sentences."

5.6 Keyword / Tag Prompts:

A list of keywords or tags that guide the output without full sentences. Used in content generation pipelines.

"Blog post | Python | beginners | 200 words | friendly tone"

5.7 Template Prompts:

Reusable prompt structures with placeholders filled at runtime. Common in automated pipelines and customer support bots.

"Write a {tone} email to {recipient} about {topic} in {word_count} words."

5.8 Constraint Prompts:

Add explicit constraints to limit or shape output.

"Explain quantum entanglement. No jargon. Under 50 words."

5.9 Continuation Prompts:

Give the start of a text and ask the model to continue.

"Once upon a time, in a world where AI governed cities..."

5.10 Negative Prompts:

Specify what the model should NOT do or include. Useful for filtering unwanted content or formats.

"Explain machine learning. Do not use code examples. Do not use the word algorithm."

6. Prompt Resolution, Bias Parameters & Support Parameters

6.1 Prompt Resolution:

Prompt resolution refers to how a language model interprets, processes, and ultimately responds to a given prompt. It involves several internal steps:

- **Tokenisation:** The prompt text is split into tokens (sub-words or characters) that the model can process.
- **Contextual Embedding:** Each token is converted into a high-dimensional vector representing its meaning in context.
- **Attention Mechanism:** The model determines which parts of the prompt are most relevant to each other using self-attention.
- **Decoding:** The model generates output tokens one by one, each conditioned on all previous tokens (autoregressive generation).
- **Post-processing:** The raw token output is decoded back to human-readable text and optionally filtered.

Prompt resolution can fail when prompts are ambiguous, too long (exceeding the context window), contain conflicting instructions, or include out-of-distribution vocabulary the model was not trained on.

6.2 Bias Parameters:

Bias in the context of language model parameters refers to the tendency of the model to favour certain outputs, patterns, or viewpoints. Bias can originate from training data, model architecture, or inference-time settings.

Types of Bias in LLMs:

- **Training Data Bias:** If training data over-represents certain demographics, viewpoints, or languages, the model may favour them in its outputs.
- **Positional Bias:** Models sometimes favour content at the beginning or end of the context window (primacy/recency bias).
- **Verbosity Bias:** Models may prefer longer, more detailed responses even when brevity is requested.
- **Sycophancy Bias:** Models may agree with the user's stated opinion rather than providing an objective answer.
- **Frequency Bias:** Common patterns in training data are reproduced more often than rare but correct alternatives.

Inference-Time Bias Controls:

- **Logit Bias:** Adjusts the probability of specific tokens appearing in output. A positive value increases likelihood; negative reduces it. Used to suppress unwanted words.
- **Presence Penalty:** Penalises tokens that have already appeared in the output, encouraging more diverse vocabulary.
- **Frequency Penalty:** Penalises tokens proportionally to how often they have already appeared, further reducing repetition.

6.3 Support Parameters (Inference / Sampling Parameters):

These parameters control how the model samples its output tokens and shape the creativity, determinism, and length of responses:

- **Temperature:** Controls randomness. Range: 0.0 to 2.0. Low (0.0–0.3): deterministic, focused, factual. High (0.8–1.5): creative, diverse, unpredictable. Default is usually 1.0.
- **Top-p (Nucleus Sampling):** At each step, only tokens whose cumulative probability reaches p are considered. E.g. top_p=0.9 means the model samples from the top 90% probability mass. Reduces low-probability hallucinations.

- **Top-k:** Limits sampling to the k most probable next tokens at each step. E.g. top_k=40 means only the 40 most likely tokens are candidates.
- **Max Tokens:** Hard limit on the number of tokens the model can generate in a response. Controls cost and prevents runaway outputs.
- **Stop Sequences:** One or more strings that, when generated, cause the model to stop output immediately. Useful for structured generation (e.g. stop at newline or closing bracket).
- **Stream:** Boolean flag. When true, tokens are streamed back incrementally as they are generated, rather than waiting for the full response. Improves perceived latency.
- **Seed:** Sets the random seed for reproducibility. Given the same prompt and seed, the model produces the same output (when temperature > 0, results may still vary slightly).
- **System Prompt:** A special input (supported by Claude, GPT-4, etc.) that sets persistent instructions, persona, and constraints. Applied before user messages. Not directly visible to end users.

This guide covers all seven topics: API fundamentals, local variables, authentication types, OAuth 2.0, prompting techniques, short prompt types, and inference/bias parameters. Suitable for developers, students, and AI practitioners.